ASSESSMENT TOOL 2 –INPUT

NAME : RAGAVI S
REGISTER NO : 192565027
COURSE CODE : CSA1717
COURSE NAME : ARTIFICIAL INTELLIGENCE

QUESTION 1: Constraint Satisfaction Problem (Doctor Shift Scheduling)

INPUT :

```
variables = ['D1', 'D2', 'D3']
domains = {
    'D1': ['Morning', 'Afternoon'],              # Constraint i: D1 != Night
    'D2': ['Morning', 'Afternoon', 'Night'],
    'D3': ['Afternoon', 'Night']                 # Constraint iv: D3 != Morning
}

shift_order = {'Morning': 1, 'Afternoon': 2, 'Night': 3}

def is_consistent(assignment, var, value):
    # Constraint iii: Only one doctor per shift (All-different)
    if value in assignment.values():
        return False, "Violation: Shift already assigned to another doctor"

    temp_assignment = assignment.copy()
    temp_assignment[var] = value

    # Constraint ii: D2 must work before D3 (Morning < Afternoon < Night)
    if 'D2' in temp_assignment and 'D3' in temp_assignment:
```

```
        if shift_order[temp_assignment['D2']] >= shift_order[temp_assignment['D3']]:
            return False, "Violation: D2 must work before D3"

    return True, "Valid"

def backtrack(assignment, step=1):
    if len(assignment) == len(variables):
        return assignment

    var = variables[len(assignment)]
    print(f"\n[Step {step}] Attempting assignments for variable: {var}")

    for val in domains[var]:
        consistent, reason = is_consistent(assignment, var, val)
        print(f"  -> Testing {var} = {val}: {reason}")

        if consistent:
            assignment[var] = val
            result = backtrack(assignment, step + 1)
            if result is not None:
                return result
            del assignment[var]  # Backtrack

    return None

print("=" * 65)
print("QUESTION 1: Doctor Shift Scheduling (Backtracking Search)")
print("=" * 65)

solution = backtrack({})
```

```
print("\n--- FINAL SCHEDULING RESULT ---")
if solution:
    for doc, shift in solution.items():
        print(f"  {doc} -> {shift} Shift")
else:
    print("  No valid assignment found.")
```

QUESTION 2: Grid Navigation using Best-First / A* Search

INPUT :

```
# QUESTION 2: Grid Navigation (A* Search with Manhattan Distance)

import heapq


grid_size = (5, 5)

start = (0, 0)

goal = (4, 4)

obstacles = {(1, 1), (2, 1), (3, 1)}  # Standard obstacle block


def manhattan_distance(p1, p2):

    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])


print("=" * 65)

print("QUESTION 2: 5x5 Grid Navigation (A* Search)")

print("=" * 65)
```

```
# Priority Queue stores: (f_score, g_score, current_node, path)
open_list = []
heapq.heappush(open_list, (manhattan_distance(start, goal), 0, start, [start]))

visited = set()
iteration = 1

print(f"Start Node: {start}, Goal Node: {goal}")
print(f"Obstacles: {obstacles}\n")

while open_list:
    f, g, current, path = heapq.heappop(open_list)

    print(f"[Iteration {iteration}] Expanded Node: {current} | g={g}, h={f-g}, f={f}")

    if current == goal:
        print("\n--- GOAL REACHED ---")
        print(f"Optimal Path: {' -> '.join(str(p) for p in path)}")
        print(f"Total Optimal Cost: {g}")
        break
```

```
visited.add(current)

# Valid directions: Up, Down, Left, Right
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

for dr, dc in directions:
    neighbor = (current[0] + dr, current[1] + dc)

    if 0 <= neighbor[0] < grid_size[0] and 0 <= neighbor[1] < grid_size[1]:
        if neighbor not in obstacles and neighbor not in visited:
            g_new = g + 1
            h_new = manhattan_distance(neighbor, goal)
            f_new = g_new + h_new
            heapq.heappush(open_list, (f_new, g_new, neighbor, path + [neighbor]))
            print(f"  -> Generated Neighbor: {neighbor} (g={g_new}, h={h_new}, f={f_new})")

iteration += 1
```

QUESTION 3: Autonomous Rescue Robot in Partially Observable Environment

INPUT:

```
start = (0, 0)

goal = (3, 3)


# Dynamic environment layout detected by sensors

obstacles = {(1, 0), (1, 1)}

risky_zones = {(0, 2), (2, 2)}  # Standard cost 1 + Risky cost 2 = Total cost 3


current_state = start

visited_costs = {start: 0}

path_history = [start]

total_cost = 0


print("=" * 65)

print("QUESTION 3: Rescue Robot in Partially Observable Grid (Online UCS)")

print("=" * 65)

print(f"Start Position: {start}, Survivor Position: {goal}")


while current_state != goal:
```

```
print(f"\n[Current Position]: {current_state} | Cumulative Cost: {total_cost}")


# Sensor perception step: Scan adjacent cells
r, c = current_state
candidates = [(r-1, c), (r+1, c), (r, c-1), (r, c+1)]


valid_moves = []
for n in candidates:
    # Grid boundary check (4x4 environment)
    if 0 <= n[0] < 4 and 0 <= n[1] < 4:
        if n not in obstacles:
            step_cost = 3 if n in risky_zones else 1
            valid_moves.append((n, step_cost))


# Filter unvisited moves
unvisited_moves = [(n, c_cost) for n, c_cost in valid_moves if n not in visited_costs]


if unvisited_moves:
    # Decision step: Choose action with lowest step cost
    unvisited_moves.sort(key=lambda x: x[1])
    next_state, move_cost = unvisited_moves[0]
```

```
            total_cost += move_cost
            visited_costs[next_state] = total_cost
            path_history.append(next_state)
            current_state = next_state
            print(f"  -> Moving to {next_state} (Step Cost: {move_cost})")
        else:
            # Backtrack if facing a dead end
            print("  -> Dead end encountered! Backtracking...")
            path_history.pop()
            if not path_history:
                print("No path available!")
                break
            current_state = path_history[-1]

    if current_state == goal:
        print("\n--- RESCUE OPERATION COMPLETE ---")
        print(f"Path Traversed: {' -> '.join(str(p) for p in path_history)}")
        print(f"Total Path Cost: {total_cost}")
```